



AULA 1

Introdução a Python e orientação a objetos



A PROFESSORA

Tatiana Escovedo

Professora do Departamento de Informática da PUC-Rio, onde coordena cursos de pós-graduação *lato sensu* e colabora com pesquisas nas áreas de ciência de dados e engenharia de *software*. Atua como gerente da área de tecnologia, gestão de dados e conhecimento e na Diretoria de Comercialização e Logística da Petrobras. É doutora em engenharia elétrica na área de métodos de apoio à decisão e mestre em informática na área de engenharia de *software* pela PUC-Rio, e autora dos livros *Introdução a Data Science - Algoritmos de Machine Learning* e *Métodos de Análise, Jornada Java e Jornada Python*.



Objetivos de aprendizagem da aula

Ao final desta aula, você irá:

- compreender conceitos básicos da linguagem Python;
- entender trechos de código simples em Python;
- entender conceitos básicos de orientação a objetos (classes, objetos, variáveis e métodos).

Nesta primeira aula, teremos o primeiro contato com a linguagem Python e com a Programação Orientada a Objetos (POO).

Que história é essa?

Para iniciar nossa conversa, vamos conhecer um pouco da história da linguagem Python?

A linguagem Python foi criada na década de 1980 por Guido Van Rossum, com o intuito de ser uma linguagem de programação interpretada e com comandos simples e fáceis de entender. Além disso, sua escrita deveria ser a mais próxima possível do inglês. Dessa forma, pode-se dizer que a linguagem Python tem uma curva de aprendizado bem reduzida quando comparada com outras linguagens de programação, como Java ou C#.

Interessante, não é mesmo?

Fique atento!

Para estudar os principais conceitos da linguagem Python, vamos examinar alguns trechos de códigos-fonte.

Introdução à linguagem Python

Um programa em Python pode ser um único arquivo com a extensão .py ou uma pasta com várias subpastas e diversos arquivos contendo código Python e outras informações relevantes ao programa, como *datasets* e imagens.

Ao iniciar o aprendizado de uma nova linguagem de programação, a forma mais comum de escrever o primeiro programa é imprimir uma mensagem na tela, como a clássica *“Hello, world!”*.

Em nosso primeiro programa, vamos imprimir uma mensagem na saída-padrão, também denominada *console* ou linha de comando.

Utilize a função *print()*, com o texto a ser impresso na saída-padrão dentro dos parênteses e entre aspas. Em seguida, execute o código, conforme o exemplo.

cod.py

```
1 # Isto é um comentário de apenas uma linha
2 print("Hello, World!")
```

Hello, World!

É importante ficar atento a algumas observações. Clique nos *cards* para saber mais.

Parênteses

P
e v

Interativo

Descrição do interativo

Agora, você vai conhecer alguns conceitos básicos e muito importantes da linguagem Python.

Variáveis

As variáveis armazenam valores de determinados tipos para serem manipulados em nossos programas.

Fique atento!

Os principais tipos de variáveis simples, em Python, são: *int* (inteiro), *float* (real), *bool* (booleano, podendo assumir os valores *True* ou *False*) e *str* (*string*, ou texto).

Em Python, não existe um comando para declarar uma variável, que é criada no momento em que um valor é atribuído a ela. Também não é necessário declarar os tipos das variáveis, sendo possível, inclusive, convertê-las para outro tipo, posteriormente, por meio de um *casting*.

Observe alguns exemplos:

cod.py

```
1 # Variáveis: Definição e tipos
2
3 # Declarando 4 variáveis e atribuindo valores a elas
4 uma_string = "Aluno"
5 um_inteiro = 7
6 um_float = 7.584
7 um_booleano = True
8
9 # Imprimindo o valor e o tipo de uma das variáveis
10 print(um_float)
11 print(type(um_float))
```

```
7.584
<class 'float'>
```

cod.py

```
1 # Casting: Convertendo a variável do tipo float para int e para string
2 print(int(um_float))
3 print(str(um_float))
```

```
7
7.584
```

É possível, ainda, trabalhar com operações matemáticas em Python, usando os operadores básicos soma (+), subtração (-), multiplicação (*), divisão (/), potenciação (**) e resto da divisão (%). Veja alguns exemplos:

cod.py

```
1 # Operações Aritméticas
2
3 # Soma, Subtração, Multiplicação e Divisão
4 resultado_1 = 7 + 5
5 resultado_2 = um_inteiro - resultado_1
6 resultado_3 = resultado_2 * 3
7 resultado_4 = resultado_3 / 4
8
9 # Imprimindo os resultados concatenando uma string com o resultado numérico
10 print("Resultado da soma = " + str(resultado_1))
11 print("Resultado da subtração = " + str(resultado_2))
12 print("Resultado da multiplicação = " + str(resultado_3))
13 print("Resultado da divisão = " + str(resultado_4))
14
15 # Potenciação e Resto da Divisão
16 resultado_5 = 2 ** 4
17 resultado_6 = 13 % 4
18
19 # Imprimindo os resultados concatenando uma string com o resultado numérico
20 print("Resultado da potenciação = " + str(resultado_5))
21 print("Resultado do resto da divisão = " + str(resultado_6))
```

```
Resultado da soma = 12
Resultado da subtração = -5
Resultado da multiplicação = -15
Resultado da divisão = -3.75
Resultado da potenciação = 16
Resultado do resto da divisão = 1
```

cod.py

```
1 # Incremento e Decremento
2
3 # Decrementando 3 unidades
4 um_inteiro -= 3
5 print("Resultado: " + str(um_inteiro)) # 7 - 3 = 4 --> agora um_inteiro é 4
6
7 # Incrementando 5 unidades
8 um_inteiro += 5
9 print("Resultado: " + str(um_inteiro)) # 4 + 5 = 9 --> agora um_inteiro é 9
```

```
Resultado: 4
Resultado: 9
```

Já os operadores de igualdade permitem comparar se duas variáveis ou valores são iguais (==) ou diferentes (!=). Os operadores de comparação (>, <, >= e <=) também são muito utilizados para comparar variáveis numéricas. Há, ainda, os operadores lógicos *and* (e) e *or* (ou).

As possíveis respostas para os operadores de igualdade e comparação são os valores booleanos *True* (verdadeiro) ou *False* (falso). Veja mais alguns exemplos:

<>

cod.py

```
1 # Operadores de Igualdade
2
3 variavel_um = 1
4 variavel_dois = 2
5 variavel_tres = 3
6
7 print(1 == variavel_um)
8 print(2 == variavel_um)
9
10 equal = (variavel_um == variavel_dois) # os () são opcionais
11 # mas ajudam a legibilidade
12 not_equal = (variavel_um != variavel_dois)
```

Interativo

Descrição do interativo

Strings

As *strings* armazenam uma ou mais palavras em texto. Existem diversas operações úteis com *strings*, como:

- concatenação de duas ou mais *strings*;
- verificação do tamanho de uma *string*;
- operações envolvendo indexação e *substrings*;
- multiplicação de uma *string* por um número;
- operador *in*;
- *escaping*;
- conversão para maiúsculas e minúsculas;
- formatação.

Clique nos cards para ver

Concatenação

Tamanho

Interativo

Descrição do interativo

Coleções

Além dos tipos simples, como *int* e *float*, a linguagem Python também fornece algumas estruturas de dados compostas ou que contenham outros tipos de dados, chamadas estruturas contêiner. Listas, tuplas e dicionários são exemplos dessas estruturas.



Clique para conhecer cada uma delas:

- Listas** 
- Tuplas** 
- Dicionários** 

Veja alguns exemplos a seguir:

Tipos de coleções



Listas

```
cod.py
1 # Listas
2
3 numeros = [1, 10, 100, 1000]
4 print(numeros)
5
6 # imprimir o elemento de índice 2 - inicia no 0
7 print(numeros[2])

[1, 10, 100, 1000]
100
```

```
cod.py
1 # Operações de Listas
2
3 # incluir 2 itens na lista usando +=
4 numeros += [10000, 100000]
5 print(numeros)
6
7 # incluir 1 item na lista usando append
8 numeros.append(1000000)
9 print(numeros)
10
11 # substituir os itens nas posições 1 e 2 por 7
12 # [índice_incluído:índice_excluído]
13 numeros[1:3] = [7]
14 print(numeros)
15
16 # remover os itens nas posições 1 e 2 da lista
17 numeros[1:3] = []
18 print(numeros)
19
20 # tamanho da lista
21 print(len(numeros))

[1, 10, 100, 1000, 10000, 100000]
[1, 10, 100, 1000, 10000, 100000, 1000000]
[1, 7, 1000, 10000, 100000, 1000000]
[1, 10000, 100000, 1000000]
4
```



Tuplas

```
cod.py
1 # Tuplas
2
3 naipes = ('copas', 'ouros', 'espadas', 'paus')
4 print(naipes)

('copas', 'ouros', 'espadas', 'paus')
```



Dicionários

```
cod.py
1 # Dicionários
2
3 # criar e imprimir um dicionário
4 notas = {"Ana": 8, "Maria": 5, "Thais": 10}
5 print(notas)
6
7 # acessar o valor correspondente à chave "Thais"
8 print(notas["Thais"])
9
10 # incluir novo item
11 notas["Zaira"] = 9
12 print(notas)
13
14 # remover item
15 del notas["Thais"]
16 print(notas)
17
18 # checar se notas contém o item "Maria"
19 print("Maria" in notas)

{'Ana': 8, 'Maria': 5, 'Thais': 10}
10
{'Ana': 8, 'Maria': 5, 'Thais': 10, 'Zaira': 9}
{'Ana': 8, 'Maria': 5, 'Zaira': 9}
True
```



Condições

Em programação, é muito comum o usuário querer executar um comando apenas se determinada condição for atendida. Para isso, há os operadores *if* (se), *else* (senão) e *elif* (senão-se), que sempre retornam um resultado booleano (*True* ou *False*) e podem ser utilizados em conjunto com operadores de igualdade, lógicos e de comparação.

Acompanhe os exemplos e entenda melhor:

Comando

Interativo

Descrição do interativo

Loops

Quando há a necessidade de executar um comando **até que** ou **enquanto** determinada condição seja atendida, é possível usar os operadores *for* e *while*, que permitem fazer um *loop*.

Pode-se, ainda, adicionalmente, utilizar os comandos *break* e *continue*.
Observe alguns exemplos a seguir:

for

Saiba mais

Interativo

Descrição do interativo

Funções

As funções possibilitam ao desenvolvedor concentrar em um único lugar um trecho de código que pode ser chamado várias vezes ao longo do programa, facilitando a manutenção da aplicação. As funções podem retornar algo e/ou receber argumentos quando chamadas, bem como declarar parâmetros *default*.

Conheça alguns exemplos a seguir:

Exemplos de funções



Funções

```
cod.py
1 # Funções
2
3 # definir uma função chamada hello_world
4 def hello_world():
5     print("Hello, World!")
6     print("Oi, Mundo!")
7     print("Salut, Monde!")
8     print("Hola, Mundo!")
9
10 # chamar a função 3 vezes
11 for i in range(3):
12     hello_world()

Hello, World!
Oi, Mundo!
Salut, Monde!
Hola, Mundo!
Hello, World!
Oi, Mundo!
Salut, Monde!
Hola, Mundo!
Hello, World!
Oi, Mundo!
Salut, Monde!
Hola, Mundo!
```



Funções com parâmetros

```
cod.py
1 # Funções com parâmetros
2
3 # x é um parâmetro
4 def uma_funcao(x):
5     print("x = %d" % x)
6
7 # passar 5 para a função. Aqui, 5 é um argumento passado para a função
8 uma_funcao(5)

x = 5
```



Funções com retorno

```
cod.py
1 # Funções com retorno
2
3 # função que retorna a soma de dois números
4 def soma(a, b):
5     return a + b
6
7 c = soma(7, 5)
8 print("c = %d" % c)

c = 12
```



Funções com parâmetros default

```
cod.py
1 # Função com parâmetros default (padrão)
2
3 def multiplica(a, b=2):
4     return a * b
5
6 print(multiplica(5, 84))
7
8 # como b tem um valor padrão, podemos passar apenas um argumento
9 print(multiplica(7))

420
14
```



Que tal ver como aplicar esses conceitos na prática?

Criando os primeiros programas em Python

Neste vídeo, você vai ver exemplos comentados de código em Python que ilustram alguns conceitos apresentados até aqui, em forma de pequenos programas.



Assista o vídeo

Essa linguagem possibilita trabalhar rapidamente e integrar sistemas de forma eficiente. Esses conceitos são muito importantes para sua base. Não deixe de estudá-los.

Referências

Clique aqui para acessar as referências e créditos desta aula.

